PER2025-053

Exploitation d'un **driver vulnérable** sur Windows dans le cadre du **hacking éthique**

1. Identification du projet..... 2
2. Présentation du sujet.....3
2.1 Contexte et Problème.....3
2.2 Périmètre et Objectifs.....4
3. Présentation des solutions et démarche retenue..... 5
3.1 Espace des solutions possibles..... 5
3.2 Solution retenue : Une approche hybride et progressive..... 6
4. Positionnement par rapport à l'existant (État de l'art).....6
4.1 Les précédents : une menace avérée et documentée..... 6
4.2 L'approche pédagogique : le rôle d'HEVD..... 6
4.3 Les protections actuelles de l'écosystème Windows.....7
4.4 Spécificité du projet : ROP Chains et manipulation de registres MSR.....7
5. Travail réalisé : Processus et Produit..... 8
5.1 Compréhension de l'exploitation du Lenovo..... 8
5.2 HEVD et problèmes rencontrés.....11
5.3 Adaptation du code de l'exploitation.....11
5.4 Compétences techniques requises et Notions à connaître……..17
6. Conclusions et Perspectives..... 19
6.1 Bilan des résultats..... 19
6.2 Prise de recul..... 20
6.3 Perspectives (Améliorations et travaux futurs).....20
7. Références Bibliographiques..... 21

1. Identification du projet

Encadrants: Yves ROUDIER**,** Florian ECARD

Étudiants:

- **Yohan MAZZI** : PeiP (2021-2023 à Polytech Nice Sophia) puis SI mineure Cybersécurité (2023-2026 à Polytech Nice Sophia)
- **Mathias SANTOS REIS** : CPGE PT (2021-2023 au lycée Rouvière à Toulon) puis SI mineure Cybersécurité (2023-2026 à Polytech Nice Sophia)
- **Romain CHARDINY** : PeiP (2021-2023 à Polytech Nice Sophia) puis SI mineure Cybersécurité (2023-2026 à Polytech Nice Sophia)
- **Ilias MOUTAOUKIL** : CPGE MPI (2021-2023 au lycée Montaigne à Bordeaux) puis SI mineure Cybersécurité (2023-2026 à Polytech Nice Sophia)

Type de projet: Développement

Date: du **17 octobre 2025** au **27 février 2026**

2. Présentation du sujet

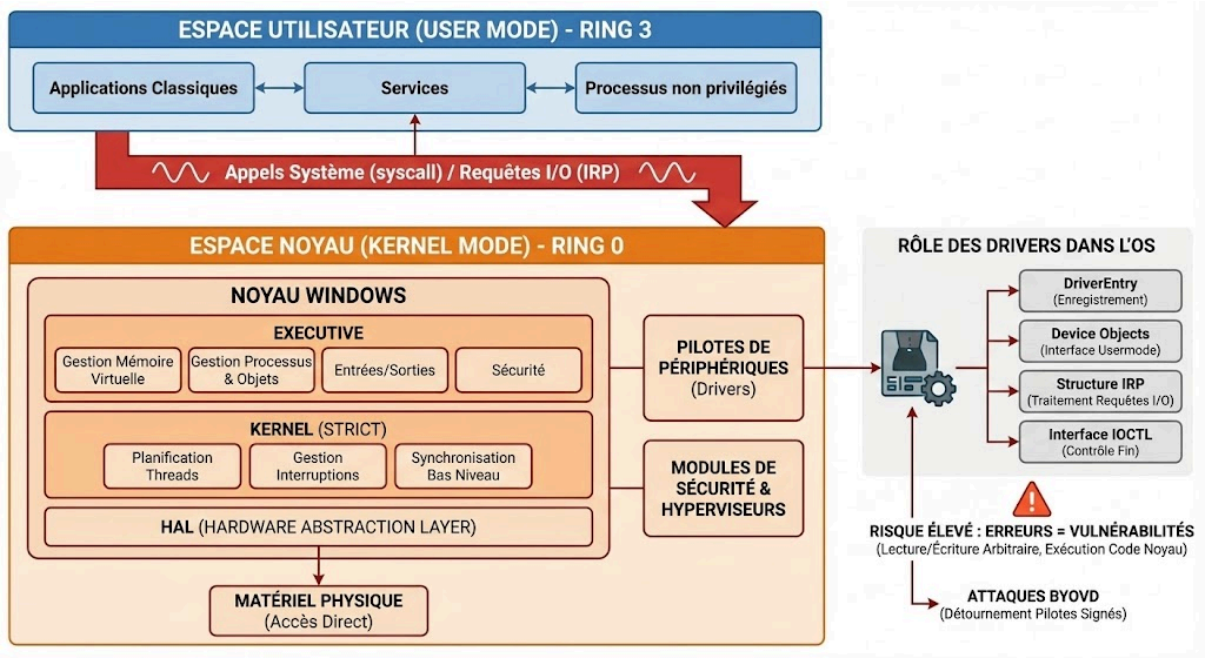
2.1 Contexte et Problème

Architecture Windows et Rôle des Pilotes

Le système d'exploitation Windows repose sur une **architecture hybride** structurée autour de deux espaces d'exécution distincts afin de garantir la stabilité et la sécurité du système : l'espace utilisateur (**User Mode** ou Ring 3) et l'espace noyau (**Kernel Mode** ou Ring 0).

Les applications classiques et les processus non privilégiés s'exécutent en **User Mode** avec des **droits strictement limités**. À l'inverse, le **mode noyau** dispose d'un **accès absolu** et sans restriction à la mémoire physique, aux périphériques matériels ainsi qu'aux registres processeur sensibles (tels que les registres MSR).

Les **pilotes** de périphériques (drivers) s'intègrent directement dans cette architecture en **mode noyau**. Parce qu'ils font le pont entre le système d'exploitation et le matériel, ils sont exécutés avec un **niveau de privilège maximal**, équivalent à celui du noyau lui-même. De ce fait, toute erreur d'implémentation ou de gestion d'accès au sein d'un pilote expose l'ensemble du système à des **vulnérabilités critiques** (lecture/écriture arbitraire, exécution de code en Ring 0).



Description de l'architecture Windows

La Menace BYOVD (Bring Your Own Vulnerable Driver)

Pour protéger le noyau contre l'exécution de code malveillant, Microsoft impose un mécanisme strict appelé ***Driver Signature Enforcement*** (DSE), qui exige que tout pilote chargé en Ring 0 soit signé cryptographiquement par une autorité reconnue.

Cependant, le modèle d'attaque **BYOVD** exploite précisément cette chaîne de confiance. L'attaque consiste, pour un acteur malveillant, à importer sur le système cible un **pilote tiers légitime**, correctement signé, mais **comportant des vulnérabilités** connues (comme un manque de contrôle d'accès sur ses IOCTL). En détournant ce pilote signé, l'attaquant **contourne le mécanisme DSE** et **obtient des primitives d'exploitation** lui permettant d'exécuter du code arbitraire avec les privilèges du noyau.

Enjeux Défensifs

Comprendre et maîtriser ces mécanismes d'exploitation est aujourd'hui crucial pour la cybersécurité défensive. En effet, les attaques de type BYOVD sont particulièrement redoutables car le contrôle du noyau permet aux attaquants de désactiver de manière furtive les mécanismes de sécurité en place, tels que les solutions EDR (Endpoint Detection and Response), les antivirus ou les processus protégés (PPL).

2.2 Périmètre et Objectifs

Cadre Éthique

Ce projet s'inscrit dans un cadre strictement **académique**, dans un but exclusivement **pédagogique** et de recherche en cybersécurité. Afin de prévenir tout risque de dérive, les expérimentations sont confinées à un **environnement de laboratoire** (machines virtuelles isolées) et n'impliquent aucun développement d'outils à usage malveillant externe.

Scope Technique

L'étude et la reproduction de l'attaque s'appuient sur l'infrastructure technologique suivante :

- **Environnement cible :** Windows 11 (exécuté en machine virtuelle sécurisée), Windows 10 (sur VM également).
- **Outils de développement :** Visual Studio 2022, Windows SDK, et le kit de développement de pilotes WDK.
- **Outils d'analyse et de débogage :** WinDbg (débogage noyau) ainsi que des outils d'analyse binaire pour l'étude statique (IDA par exemple).

Objectif Principal

L'objectif fondamental du projet est de réaliser de manière contrôlée une preuve de concept (PoC) utilisant un pilote Windows vulnérable existant. Il s'agit de reproduire l'intégralité d'un scénario BYOVD, de l'analyse binaire du pilote jusqu'à son exploitation effective, afin d'observer les mécanismes internes du noyau, d'évaluer concrètement les risques associés et d'étudier les possibilités de détection et de mitigation.

3. Présentation des solutions et démarche retenue

3.1 Espace des solutions possibles

Afin de répondre à notre besoin d'étudier les vulnérabilités des pilotes Windows et de comprendre les mécanismes d'abus en mode noyau, l'espace des solutions a été exploré à travers trois approches distinctes :

- **Développer notre propre driver vulnérable :** Cette méthode consistait à concevoir un pilote de toutes pièces, en y intégrant volontairement des failles (identiques à celles présentes dans le driver de la CVE).
    - *Avantage :* Elle offre une maîtrise absolue du code source et facilite la compréhension des mécanismes de base.
    - *Inconvénient :* Elle manque de réalisme, car elle ne confronte pas l'équipe aux contraintes d'un véritable scénario d'attaque BYOVD sur un système en production.
- **Utiliser un driver éducatif (HEVD) :** Le recours au *HackSys Extreme Vulnerable Driver* (HEVD) a été évalué comme un tremplin d'apprentissage.
    - *Avantage :* Disposant d'une documentation communautaire abondante, c'est l'outil de référence pour une montée en compétences initiale et ciblée sur l'exploitation noyau.
- **Exploiter une vulnérabilité réelle (CVE) :** Cette dernière approche reposait sur l'identification et l'utilisation d'un pilote public, existant, signé par un éditeur légitime, mais comportant une vulnérabilité documentée.
    - *Avantage :* Elle permet la mise en situation d'un scénario BYOVD parfaitement réaliste et représentatif des menaces actuelles.
    - *Inconvénient :* Elle implique une complexité technique particulièrement élevée, exigeant des compétences en *reverse engineering* pour appréhender la structure binaire du pilote sans disposer de son code source.

3.2 Solution retenue : Une approche hybride et progressive

Afin de concilier une courbe d'apprentissage maîtrisée avec un fort niveau de réalisme technique, nous avons choisi d'adopter une démarche hybride. L'équipe a structuré la progression en trois phases clés :

1. **Phase d'initiation :** Cette étape préparatoire a consisté à mettre en place l'environnement et à développer un pilote "Hello World" minimal. Cette étape était indispensable pour valider la chaîne de compilation, le processus de signature et les outils de débogage (WinDbg) au sein de la machine virtuelle isolée, avant toute manipulation dangereuse.
2. **Phase d'entraînement (HEVD) :** Avant de nous attaquer à une cible réelle complexe, nous avons intégré une phase d'entraînement utilisant le driver HEVD. Cette étape nous a permis d'assimiler les concepts fondamentaux de la corruption mémoire en mode noyau (tels que les *Stack Overflows* et les *ROP chains*) et de fiabiliser nos méthodes de communication via IOCTL.
3. **Phase de réalisation (BYOVD Réel) :** L'aboutissement de notre démarche a été la reproduction d'un PoC (Proof of Concept) sur une cible réelle. Nous avons sélectionné la vulnérabilité CVE-2025-8061, qui affecte le pilote légitime *LnvMSRIO.sys* distribué par Lenovo. Ce choix nous a permis d'analyser et de documenter un vecteur d'attaque BYOVD de bout en bout.

4. Positionnement par rapport à l'existant (État de l'art)

L'étude des vulnérabilités noyau et du modèle BYOVD (Bring Your Own Vulnerable Driver) s'inscrit dans un contexte où les attaques de bas niveau se multiplient pour contourner les solutions de sécurité modernes.

4.1 Les précédents : une menace avérée et documentée

Le modèle d'attaque BYOVD n'est pas qu'un concept théorique; il est activement exploité dans des campagnes malveillantes réelles. De nombreux cas documentés illustrent l'utilisation de pilotes signés pour compromettre des systèmes:

- **Slingshot APT :** Exploitation d'un driver MikroTik vulnérable pour obtenir un accès noyau persistant.
- **Ransomware BlackByte :** Utilisation de drivers MSI vulnérables dans le but spécifique de neutraliser les solutions EDR (Endpoint Detection and Response).
- **Vulnérabilités de grande ampleur :** La CVE-2021-21551 affectant les pilotes Dell (fournissant une primitive d'écriture arbitraire) ou encore l'utilisation de fonctionnalités non documentées dans les drivers GIGABYTE permettant la modification directe du noyau.

Ces exemples démontrent que le détournement de pilotes légitimes est devenu une technique standard et critique dans l'arsenal des attaquants modernes.

4.2 L'approche pédagogique : le rôle d'HEVD

Avant d'analyser des vulnérabilités complexes comme celles de Lenovo, l'état de l'art de la recherche en sécurité s'appuie souvent sur des environnements standardisés. Dans le cadre de ce projet, nous nous sommes positionnés en utilisant **HEVD (HackSys Extreme Vulnerable Driver)**. Il s'agit d'un pilote Windows intentionnellement vulnérable, conçu spécifiquement pour la formation à l'exploitation de failles noyau. L'étude de HEVD a servi de pont entre la théorie et la pratique, nous permettant d'isoler et de comprendre individuellement des mécanismes comme les Buffer Overflow, avant d'aborder les primitives de lecture/écriture arbitraires du pilote Lenovo.

4.3 Les protections actuelles de l'écosystème Windows

Pour contrer ces attaques au niveau du noyau, Microsoft a implémenté plusieurs couches de sécurité que notre attaque doit prendre en compte ou contourner :

- **DSE (Driver Signature Enforcement) :** Ce mécanisme impose que tout pilote soit signé par une autorité reconnue pour être chargé. C'est précisément le "verrou" que l'attaque BYOVD contourne en apportant un pilote déjà signé, mais faillible.
- **KASLR (Kernel Address Space Layout Randomization) :** Il place de façon aléatoire les zones de données et le code du noyau dans la mémoire virtuelle à chaque démarrage.
- **SMEP (Supervisor Mode Execution Prevention) et SMAP (Supervisor Mode Access Prevention) :** Ces protections matérielles interdisent au processeur (lorsqu'il est en Ring 0) d'exécuter du code (SMEP) ou d'accéder à des données (SMAP) situés dans des pages mémoire de l'espace utilisateur (Ring 3).
- **HVCI (Hypervisor-Protected Code Integrity) et Blocklist :** Pour contrer le BYOVD, Microsoft maintient une liste noire (Driver Blocklist) empêchant le chargement de pilotes connus comme vulnérables. De plus, HVCI utilise la virtualisation pour interdire la modification du code noyau, même en cas de compromission.

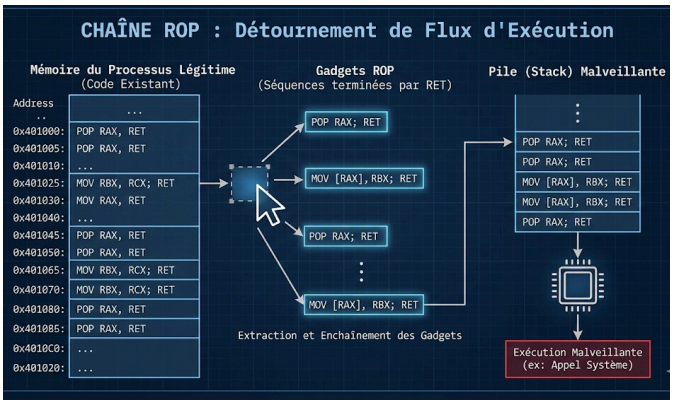
4.4 Spécificité du projet : ROP Chains et manipulation de registres MSR

Contrairement à une simple analyse théorique ou à l'exploitation d'une faille basique sur un système non protégé, la spécificité de notre solution réside dans l'implémentation d'une chaîne d'attaque complète capable de contourner dynamiquement les protections modernes (KASLR, SMEP, SMAP).

Pour exécuter notre shellcode (situé en espace utilisateur) avec les privilèges du noyau, nous ne pouvons pas simplement y faire pointer le flux d'exécution à cause de SMEP. Notre projet s'appuie donc sur des techniques d'exploitation avancées :

- **Le Leak d'informations :** L'utilisation de l'instruction *__rdmsr* via le pilote vulnérable nous permet de lire le registre MSR *LSTAR* (contenant l'adresse de `KiSystemCall64`) et ainsi de calculer l'adresse de base du noyau pour vaincre kASLR.
- **La ROP Chain (Return-Oriented Programming) :** Pour désactiver SMEP et SMAP, nous avons dû construire une chaîne ROP. Cette technique consiste à enchaîner l'exécution de petits bouts de code légitimes déjà présents dans le noyau (les

"gadgets"). En utilisant des adresses absolues de gadgets tels que `swapgs`, `iretq`, et `mov cr4, rcx` , notre attaque modifie temporairement le registre processeur



CR4 (pour désactiver SMEP) et le registre RFLAGS (pour désactiver SMAP).

<u>Illustration d'une chaîne ROP</u>

C'est cette ingénierie minutieuse (Hooking du LSTAR couplé à une ROP Chain pour abaisser les défenses avant le vol de token) qui positionne ce travail bien au-delà d'une simple preuve de concept, en démontrant l'impact critique des interactions *User vers Kernel* mal validées.

5. Travail réalisé : Processus et Produit

5.1 Compréhension de l'exploitation du Lenovo

Résumé de l'exploitation:

Le driver initialise un device avec un lien symbolique nommé **WinMsrDev** (initialisation dans *DriverEntry → DriverSetup*). Aucun contrôle d'accès n'est appliqué sur ce device : tout programme peut donc communiquer avec lui.

Quatre fonctions (*IoControlCode*) exposées via le handler IOCTL du driver semblent exploitables.

La fonction **IoControlCode 0x9c406104** permet une lecture de la mémoire physique en exploitant un appel non sécurisé à *MmMapIoSpace()*. Une adresse physique et une taille sont fournies à *MmMapIoSpace*, qui mappe cette mémoire dans l'espace virtuel du noyau et renvoie un pointeur virtuel vers la mémoire physique. La fonction *memcpy_wrapper()* copie ensuite les données depuis la mémoire mappée vers l'*OutputBuffer*.

La fonction **IoControlCode 0x9c40a108** permet une écriture en mémoire physique, toujours via un appel non sécurisé à *MmMapIoSpace()*. Cette fois, le tampon de destination de *memcpy_wrapper()* correspond à la région mémoire mappée par *MmMapIoSpace*, tandis que le tampon source est contenu dans l'*InputBuffer*.

La fonction **IoControlCode 0x9c402084** permet la lecture des registres MSR. L'instruction *__rdmsr* (spécifique au mode kernel) lit un registre MSR à partir de son identifiant, puis retourne le résultat dans le buffer de sortie. La fonction accepte n'importe quel identifiant de registre MSR et ne vérifie pas les droits du processus appelant. Il est ainsi possible de récupérer l'adresse de *KiSystemCall64()* depuis le registre MSR **LSTAR**, invoqué en Ring-0 à chaque appel système effectué depuis l'espace utilisateur (Ring-3).

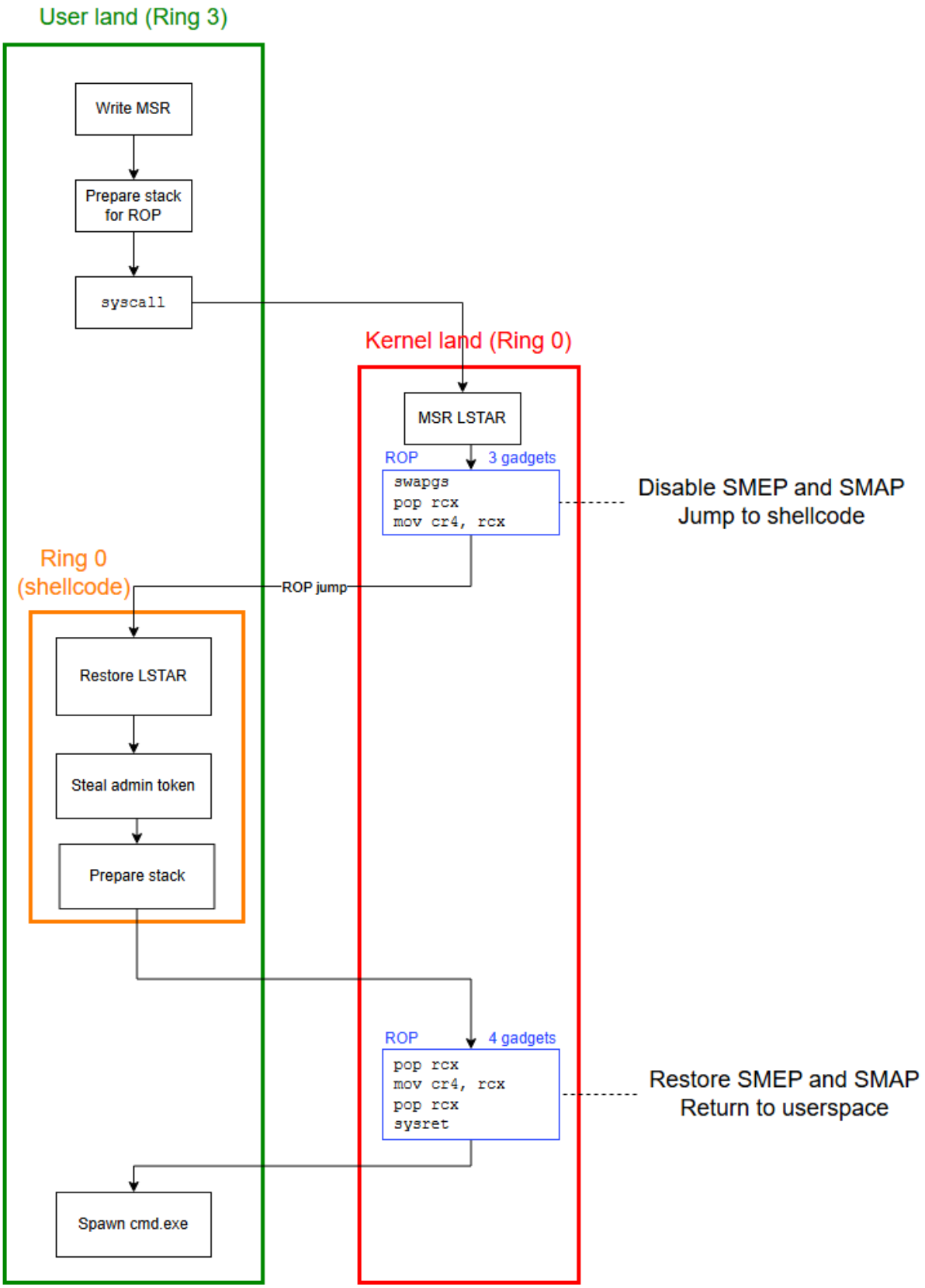
De manière similaire, **IoControlCode 0x9c402088** permet l'écriture dans les registres MSR via l'instruction *__wrmsr*.

La capacité de lecture et d'écriture dans les registres MSR permet d'écraser l'adresse contenue dans le registre MSR **LSTAR**. Lorsqu'un *syscall* est ensuite exécuté depuis l'espace utilisateur, l'adresse pointée par LSTAR correspond alors à notre shellcode, exécuté avec les privilèges kernel. Cette modification perturbe le fonctionnement normal des appels système et peut provoquer un écran bleu de la mort (BSOD). Après exécution du code, il est donc impératif de restaurer la valeur originale du registre afin de préserver la stabilité du système.

Les mécanismes **SMAP** et **SMEP** protègent respectivement l'accès (lecture/écriture) et l'exécution de code dans des pages mémoire marquées utilisateur (Ring-3). Il est donc nécessaire de les désactiver. Pour cela, on utilise *EnumDeviceDrivers()* ou la lecture du registre MSR LSTAR (contenant l'adresse de *KiSystemCall64*) afin d'obtenir l'adresse de base de *ntoskrnl.exe* et contourner **kASLR**, qui randomise l'emplacement des zones mémoire. À partir de la base du noyau, on calcule les adresses absolues de gadgets (par exemple : *swapgs ; iretq*, *mov cr4, rcx*) pour désactiver SMEP. Le bit AC du registre *RFLAGS* est positionné pour autoriser temporairement les accès mémoire en user-land et désactiver SMAP. Une chaîne de gadgets (ROP) est ensuite utilisée pour modifier le registre *CR4* et désactiver le bit SMEP, permettant l'exécution de code situé en mémoire utilisateur.

Le shellcode a pour objectif de voler le token du processus système (PID 4). Après modification du registre LSTAR pour contourner kASLR, les appels système ne fonctionnent plus, car LSTAR pointe vers une adresse invalide ; la valeur originale de *KiSystemCall64* est donc réécrite dans le registre LSTAR. La liste des processus (*EPROCESS*) est parcourue via le registre *GS* (pointant vers la *KPCR* en Ring-0) afin d'identifier le processus SYSTEM (PID = 4) et copier son token dans le processus courant. SMEP est finalement réactivé afin d'éviter d'introduire d'autres vulnérabilités.

Il devient alors possible d'exécuter des commandes avec les privilèges **NT AUTHORITY\SYSTEM** dans l'espace utilisateur (Ring-3), tant que le processus courant reste actif (création d'un utilisateur administrateur, etc.).



Les grandes étapes de l'exploitation du Lenovo

5.2 HEVD et problèmes rencontrés

L'un des principaux problèmes rencontrés concerne la **dépendance forte de l'exploitation à l'environnement cible**. Contrairement à un programme applicatif classique, une exploitation noyau repose sur des éléments bas niveau (adresses mémoire, offsets de structures, gadgets ROP, configuration matérielle) qui ne sont pas statiques.

Selon la **version** de Windows (10 ou 11), le **build** exact du système, ainsi que les **mises à jour installées**, l'organisation interne du noyau peut varier. Les offsets au sein des structures (par exemple EPROCESS), les adresses des fonctions, la disposition des sections mémoire ou encore certains mécanismes internes peuvent être modifiés. En conséquence, une exploitation fonctionnelle sur un build donné peut ne plus fonctionner sur un autre, même très proche.

Les **mécanismes de sécurité activés** constituent également un facteur déterminant. La présence ou non de protections telles que SMEP, SMAP, kASLR, VBS/HVCI ou KVA Shadow modifie directement la stratégie d'exploitation. Certaines configurations empêchent l'exécution de code en mémoire utilisateur ou rendent plus complexe la récupération d'adresses noyau, nécessitant des **adaptations spécifiques**.

Le **matériel** joue aussi un rôle important. Des différences entre **architectures processeur** (Intel vs AMD) peuvent influencer la configuration mémoire et certaines protections matérielles.

L'expérimentation avec **HEVD** (HackSys Extreme Vulnerable Driver) a notamment mis en évidence que des variations liées au CPU pouvaient entraîner des différences de comportement ou de configuration système impactant directement l'exploitation.

L'objectif du travail sur HEVD n'était pas de reproduire la forme du bug (overflow, UAF, etc.), mais d'identifier la **primitive d'exploitation finale** offerte au niveau noyau. En exploitation kernel moderne, la primitive est l'élément déterminant.

Choix de la vulnérabilité pertinente

Parmi les vulnérabilités proposées par HEVD, **Arbitrary Memory Overwrite** est celle qui se rapproche conceptuellement du cas Lenovo (CVE-2025-8061).

Cette vulnérabilité fournit directement une primitive de type :

write-what-where en Ring 0
*(Where) = *(What)*

Comme dans le driver Lenovo :

- l'IOCTL est trop permissif,
- aucune corruption préalable de pile ou de tas n'est nécessaire,

- le noyau écrit lui-même à l'adresse fournie par l'utilisateur.

Il s'agit donc d'un **exploit logique basé sur une relation de confiance violée**, et non d'une corruption mémoire accidentelle.

Notions techniques approfondies durant le travail

Le travail a nécessité la compréhension détaillée des éléments suivants :

- Architecture x64 et mécanismes de paging (CR3, PML4, hiérarchie des tables de pages).
- Fonctionnement des privilèges Ring-3 / Ring-0.
- Structure des tables de pages Windows et bits U/S, NX (XD).
- SMEP et SMAP (protections matérielles empêchant l'exécution ou l'accès à la mémoire user en mode noyau).
- kASLR (randomisation des adresses du noyau).
- KVA Shadow (séparation CR3 user/kernel mise en place après Meltdown).
- PML4 Self-Reference Entry Randomization.
- Kernel-mode Address Check (kCFG simplifié lorsque VBS/HVCI est désactivé).
- Structures internes du noyau (KPCR, KPRCB, _KPROCESSOR_STATE).
- Token stealing pour élévation de privilèges SYSTEM.

Sécurités rencontrées et contournées

L'exploitation a nécessité l'analyse et le contournement de plusieurs mécanismes :

- **SMEP** : empêche l'exécution de code situé en mémoire utilisateur.
- **kASLR** : nécessite la récupération de la base de ntoskrnl (via EnumDeviceDrivers).
- **Kernel-mode Address Check** : empêche les sauts directs vers du code user.
- **KVA Shadow** : sépare les espaces d'adressage via deux CR3 distincts.
- **PML4 self-reference randomization** : complique la manipulation directe des tables de pages.

Le contournement s’appuie sur la primitive write-what-where pour :

- modifier des entrées de tables de pages,
- altérer des pointeurs noyau,
- construire une chaîne ROP lorsque nécessaire.

Problèmes spécifiques liés au matériel (Intel vs AMD)

Un point critique identifié durant le travail concerne la dépendance au modèle micro-architectural du CPU.

L’exploit étudié repose sur des hypothèses valides principalement sur :

- processeurs Intel vulnérables à Meltdown,
- configuration Windows avec KVA Shadow activé,
- séparation stricte des CR3 user/kernel.

Or :

- Les processeurs AMD ne sont pas vulnérables à Meltdown.
- Windows n’applique pas toujours la même implémentation de KVA Shadow sur AMD.
- L’organisation des tables PML4 peut différer.
- Le comportement CR3/PCID peut changer.

Conséquence directe :
Un exploit basé sur la manipulation d’entrées PML4 ou sur des hypothèses liées au split CR3 peut fonctionner sur Intel et échouer sur AMD.

Ce n’est pas uniquement une différence de version Windows, mais une différence de **modèle de gestion mémoire induite par l’architecture CPU**.

Conclusion technique sur HEVD

Le travail sur HEVD a permis de :

- Comprendre qu’en exploitation noyau moderne, la primitive (write-what-where) est plus importante que la forme du bug.
- Analyser et contourner les protections modernes (SMEP, kASLR, KVA Shadow).

- Mettre en évidence la forte dépendance d'un exploit kernel à :
    - la version exacte de Windows,
    - la configuration de sécurité,
    - et surtout à l'architecture matérielle (Intel vs AMD).

Cela illustre pourquoi une exploitation noyau est rarement universelle et nécessite une adaptation précise à l'environnement cible.

Ainsi, une exploitation système n'est généralement pas universelle : elle est fortement dépendante d'une **combinaison précise de version, build, configuration de sécurité et matériel**. Cette variabilité rend le développement d'un **exploit noyau particulièrement complexe**, nécessitant des ajustements constants et une compréhension approfondie de l'architecture interne du système ciblé.

5.3 Adaptation du code de l'exploitation

L'exploit Lenovo étudié initialement a été développé et validé sur un environnement précis (Windows 11 24H2). Son fonctionnement repose notamment sur des adresses de gadgets ROP spécifiques ainsi que sur une valeur modifiée du registre CR4 destinée à désactiver SMEP/SMAP.

Or, dans notre environnement de test (Windows 11 25H2 – Build 26200.7623), plusieurs **adaptations** ont été nécessaires :

1. Adaptation des adresses ROP

Les **adresses des gadgets ROP** ne sont **pas universelles**. Elles dépendent directement :

- de la version de Windows,
- du build exact du noyau,
- de la version de ntoskrnl.exe,
- et de la randomisation appliquée au chargement du noyau (kASLR).

Même entre Windows 11 24H2 et 25H2, la disposition interne du noyau peut évoluer :

- déplacement de fonctions,

- modification d’offsets,
- changements d’alignement,
- recompilation avec optimisations différentes.

En conséquence, les **gadgets utilisés** dans l’exploit original (ex. mov cr4, rcx, swapgs ; iretq, etc.) n’ont **pas les mêmes adresses** dans notre build. Il a donc été nécessaire :

1. de récupérer la base de ntoskrnl.exe sur la VM cible,
2. d’identifier à nouveau les gadgets nécessaires,
3. de recalculer leurs adresses absolues,
4. et de mettre à jour la chaîne ROP en conséquence.

Cela confirme qu’un exploit noyau est fortement dépendant de la version exacte du système et qu’un simple changement de build peut invalider les hypothèses initiales.

2. Adaptation de la valeur du registre CR4

L’exploit d’origine modifie le registre CR4 afin de désactiver SMEP (et potentiellement SMAP).
Pour cela, il utilise une valeur hardcodée de CR4 dans le code.

Cependant, dans notre environnement virtualisé, cette valeur provoquait un crash immédiat de la VM. La raison est que CR4 contient plusieurs flags matériels dépendants :

- SMEP (bit 20),
- SMAP (bit 21),
- PCID,
- PAE,
- OSXSAVE,
- etc.

Certains de ces bits sont :

- dépendants du processeur,

- dépendants de l’hyperviseur,
- ou configurés dynamiquement par Windows.

La valeur hardcodée dans l’exploit original incluait des flags incompatibles avec la configuration de notre VM, ce qui entraînait une incohérence matérielle et un crash.

3. Méthodologie corrective appliquée

Pour rendre l’exploit stable dans notre environnement :

1. La valeur réelle de CR4 a été récupérée dynamiquement sur la VM.
2. Les bits correspondant à SMEP/SMAP ont été identifiés.
3. Seuls ces bits ont été retirés.
4. La nouvelle valeur résultante a été injectée dans l’exploit.

Cette approche garantit que :

- tous les autres flags matériels restent cohérents,
- la configuration spécifique au CPU/hyperviseur est respectée,
- la stabilité du système est conservée.

4. Impact de la virtualisation

Ce problème met en évidence un point important :
l’utilisation d’une machine virtuelle n’est pas neutre du point de vue micro-architectural.

Selon :

- le type d’hyperviseur,
- l’émulation ou la virtualisation des extensions CPU,
- l’exposition ou non de certains flags matériels,

La valeur effective de CR4 peut différer significativement d’un environnement à l’autre.

Ainsi, un exploit fonctionnel sur machine physique peut nécessiter des ajustements en environnement virtualisé, et inversement.

Conclusion

L'adaptation de l'exploit Lenovo a principalement porté sur :

- la mise à jour des adresses ROP, rendues invalides par le changement de build Windows,
- l'ajustement précis de la valeur du registre CR4 afin de désactiver SMEP/SMAP sans perturber les autres flags matériels,
- la prise en compte des spécificités liées à la virtualisation.

Ces modifications illustrent le **caractère non générique des exploits noyau** : leur fonctionnement dépend étroitement de la version du système, de sa configuration de sécurité et de l'architecture matérielle sous-jacente.

5.4 Compétences techniques requises et Notions à connaître

Notions techniques

Driver (pilote Windows)
Composant logiciel exécuté en mode noyau (Ring-0), généralement situé dans `C:\Windows\System32\drivers`. Il dispose de privilèges élevés et interagit directement avec le matériel et le noyau via des structures comme les IRP.

Notion de Ring (Ring-3 / Ring-0)
Modèle de privilèges processeur.

- **Ring-3** : espace utilisateur (applications).
- **Ring-0** : espace noyau (kernel), accès complet au système.
  Le passage de Ring-3 à Ring-0 s'effectue notamment via les appels système (*syscall*).

EDR (Endpoint Detection and Response)
Solution de sécurité surveillant les comportements suspects sur un poste (chargement de driver, élévation de privilèges, injections mémoire, etc.).

PPL (Protected Process Light)
Mécanisme protégeant certains processus sensibles (ex. antivirus) contre l'accès ou la modification, même avec des privilèges élevés.

Driver Signature Enforcement (DSE)
Mécanisme imposant la signature numérique des drivers pour autoriser leur chargement en mode noyau.

Local Privilege Escalation (LPE)
Technique permettant à un utilisateur disposant de droits limités d'obtenir des privilèges plus élevés (ex. SYSTEM) sur la machine locale.

IRP (I/O Request Packet)
Structure utilisée par le noyau Windows pour gérer les requêtes d'entrée/sortie entre applications et drivers. Elle transporte notamment les IOCTL vers le driver.

Adresse virtuelle du noyau Windows
Adresse située dans l'espace mémoire du noyau, distincte de l'espace utilisateur. Protégée par des mécanismes comme kASLR.

MSR (Model-Specific Register)
Registres spécifiques au processeur accessibles en mode kernel via `__rdmsr` / `__wrmsr`. Exemple : LSTAR, qui contient l'adresse de `KiSystemCall64`, point d'entrée des appels système en 64 bits.

Registre GS, swapgs et KPCR

- En Ring-0, le registre GS pointe vers la KPCR (Kernel Processor Control Region).
- L'instruction swapgs permet de basculer entre le contexte utilisateur et noyau pour accéder aux structures internes du système.

ROP (Return-Oriented Programming)
Technique d'exploitation consistant à enchaîner des fragments d'instructions existantes (*gadgets*) pour exécuter une logique arbitraire sans injecter directement un code complet.

Gadget
Courte séquence d'instructions se terminant généralement par `ret`, utilisée dans une chaîne ROP.

SMEP (Supervisor Mode Execution Prevention)
Empêche le noyau d'exécuter du code situé en mémoire utilisateur.

SMAP (Supervisor Mode Access Prevention)
Empêche le noyau d'accéder (lecture/écriture) à des pages mémoire utilisateur.

kASLR (Kernel Address Space Layout Randomization)
Randomisation des adresses du noyau afin de compliquer la prédiction des emplacements mémoire.

KVA Shadow
Mécanisme d'isolation des tables de pages noyau/utilisateur pour atténuer certaines attaques de type Meltdown.

VBS / HVCI (Virtualization-Based Security / Hypervisor-Protected Code Integrity)
Mécanismes de sécurité reposant sur l'hyperviseur pour isoler et protéger l'intégrité du code noyau.

PML4 Self-Reference Entry Randomization
Technique de randomisation liée aux structures de pagination (PML4) compliquant l'exploitation mémoire.

HalDispatchTable
Table de fonctions du Hardware Abstraction Layer historiquement ciblée pour détourner le flux d'exécution en noyau.

Compétences techniques requises

- **Reverse engineering** : analyse de binaire, compréhension du code assembleur x64, identification des IOCTL et chemins d'exécution vulnérables.

- **Écriture et lecture d'assembleur x64** : manipulation de registres (CR4, RFLAGS, MSR), compréhension des conventions d'appel et du fonctionnement des syscalls.

- **Développement C/C++ bas niveau** : interaction avec l'API Windows, manipulation de buffers, structures noyau et primitives mémoire.

- **Compréhension de l'architecture Windows interne** : EPROCESS, token, KPCR, gestion mémoire, mécanismes de protection.

- **Debug kernel (WinDbg)** : analyse de crash (BSOD), inspection mémoire, registres et structures noyau.

- **Architecture processeur x86-64** : gestion des privilèges, pagination, CR3/CR4, MSR, protections matérielles.

- **Construction de chaînes ROP** : recherche de gadgets, calcul d'adresses relatives, contournement de protections (SMEP/SMAP/kASLR).

- **Méthodologie d'exploitation** : adaptation à différentes versions/build Windows et compréhension des variations liées au matériel (Intel/AMD).

6. Conclusions et Perspectives

6.1 Bilan des résultats

- **Succès technique et élévation de privilèges :** Le principal objectif de ce projet a été atteint avec succès. En exploitant les vulnérabilités du pilote `LnvMSRIO.sys`, nous avons pu construire une chaîne d'exploitation complète contournant les

protections KASLR, SMEP et SMAP. Le shellcode final a permis de localiser le processus système en parcourant les structures `EPROCESS` et de copier son token de sécurité vers notre processus attaquant. Cela s'est traduit par l'obtention d'un shell interactif avec les privilèges maximums `NT AUTHORITY\SYSTEM` depuis un compte utilisateur standard.

- **Stabilité du système (Post-Exploitation) :** L'une des plus grandes difficultés dans le développement d'exploits en mode noyau est le maintien de la stabilité de l'OS. Nous avons réussi à restaurer l'état du système immédiatement après l'exécution du shellcode. En réécrivant la valeur originale de `KiSystemCall64` dans le registre `LSTAR` et en réactivant la protection SMEP, nous avons évité le plantage critique du système (Blue Screen of Death).

6.2 Prise de recul

- **Complexité et exigence du développement Kernel :** Ce projet a mis en lumière la rigueur extrême qu'exige le développement de bas niveau. Dans l'espace noyau (Ring 0), contrairement à l'espace utilisateur, la moindre erreur d'accès mémoire, de manipulation d'IOCTL ou d'alignement de pile ne pardonne pas et entraîne inévitablement un crash complet du système (BSOD). Bien que ce processus de débogage (via WinDbg et des machines virtuelles reliées) soit particulièrement fastidieux, il s'est révélé extrêmement formateur pour comprendre les concepts du système d'exploitation.
- **Efficacité redoutable du modèle BYOVD :** Cette expérimentation démontre de manière empirique la puissance des attaques BYOVD. Même sur un système d'exploitation Windows 11 à jour, le simple fait d'autoriser le chargement d'un pilote tiers légitimement signé, mais mal conçu, suffit à compromettre l'intégralité de la machine. La confiance aveugle accordée aux signatures cryptographiques (DSE) constitue une faille systémique majeure que les attaquants continueront d'exploiter.

6.3 Perspectives (Améliorations et travaux futurs)

Afin de prolonger cette recherche et d'approfondir l'analyse défensive et offensive, plusieurs axes d'amélioration peuvent être envisagés :

- **Confrontation avec la protection HVCI :** Il serait pertinent de tester cette même chaîne d'attaque sur un système où la fonctionnalité *Hypervisor-Protected Code Integrity* (HVCI / Memory Integrity) est activée. Théoriquement, cette protection basée sur la virtualisation devrait bloquer la modification des registres critiques (comme les MSR) ou l'exécution de code non autorisé dans le noyau. L'objectif serait d'observer le comportement de l'exploit face à ce mécanisme de défense avancé.
- **Évasion d'Antivirus et d'EDR :** Jusqu'à présent, les solutions de sécurité ont été étudiées principalement en mode "monitoring" (passif) pour observer les télémétries. Une perspective intéressante consisterait à tenter de contourner activement un antivirus ou un EDR en mode "prévention", par exemple en utilisant nos privilèges `SYSTEM` fraîchement acquis pour neutraliser ou aveugler l'agent de sécurité de l'intérieur.

- **Industrialisation et détection automatisée :** Pour adopter une approche orientée *Red Team*, il serait envisageable de développer un outil (détecteur de vulnérabilité) capable de scanner dynamiquement un système pour identifier la présence exacte de ce driver (`LnvMSRIO.sys`) ou d'autre driver avec la même vulnérabilité et de tester la disponibilité de ses interfaces IOCTL vulnérables. Cela permettrait d'automatiser et d'industrialiser le déploiement de l'exploit lors d'audits de sécurité.

7. Références Bibliographiques

Documentation Microsoft

- Documentation Windows pour les développeurs de drivers (principale page d'entrée). https://learn.microsoft.com/en-us/windows-hardware/drivers/
- Documentation de l'API pour le Kit de pilotes Windows (WDK) références détaillées des API. https://learn.microsoft.com/fr-fr/windows-hardware/drivers/ddi/
- Liste de vérification de la sécurité des pilotes — guide de bonnes pratiques pour des drivers sécurisés. https://learn.microsoft.com/fr-fr/windows-hardware/drivers/driversecurity/driver-security-checklist

Bases de données de vulnérabilités

- NIST National Vulnerability Database CVE-2025-8061 (fiche détaillée de la vulnérabilité étudiée). https://nvd.nist.gov/vuln/detail/CVE-2025-8061
- MITRE CVE List CVE repository (catalogue général des vulnérabilités). https://learn.microsoft.com/fr-fr/windows-hardware/drivers/driversecurity/threat-modeling-for-drivers
- MITRE CWE Common Weakness Enumeration (classification des types de failles). https://cwe.mitre.org/

Analyses de pilotes vulnérables

- JVNTA#90371415 Multiple third-party kernel drivers for Windows vulnerable to improper access control on IOCTL (analyse vulnérabilités IOCTL). https://jvn.jp/en/ta/JVNTA90371415/index.html

- Rapport de Kaspersky sur l'augmentation des attaques utilisant des pilotes vulnérables (contexte BYOVD attaques récents). https://www.kaspersky.fr/about/press-releases/kaspersky-detecte-une-augmentation-de-23-des-attaques-visant-les-pilotes-windows-vulnerables

- Qilin Ransomware BYOVD case study (exemple d'exploitation BYOVD dans la nature). https://cyberveille.esante.gouv.fr/actualites/qilin-ransomware-la-menace-du-bring-your-own-vulnerable-driver-2025-08-12

- A. Monzani, A. Parata, A. Oliveri, S. Aonzo, D. Balzarotti, et A. Lanzi, “Unveiling BYOVD Threats: Malware’s Use and Abuse of Kernel Drivers” dans Proceedings of the Network and Distributed System Security (NDSS) Symposium, San Diego, CA, USA, 2026. doi: 10.14722/ndss.2026.231491.

Blogs techniques de référence

- Quarkslab Analyse technique de CVE-2025-8061 (article de référence utilisé pour ton étude). https://blog.quarkslab.com/exploiting-lenovo-driver-cve-2025-8061.html
- Github - symeonp/Lenovo-CVE-2025-8061 (POC du CVE-2025-8061) https://github.com/symeonp/Lenovo-CVE-2025-8061
- Exploiting WRMSR in vulnerable drivers https://idafchev.github.io/blog/wrmsr/
- Blog sur comment fonctionne le rop et un exemple d’exploitation de programme sur windows 64bits https://www.dailysecurity.fr/return_oriented_programming/ https://www.dailysecurity.fr/windows_exploit_64_bits_rop/
- Return Oriented Programming https://ctf101.org/binary-exploitation/return-oriented-programming/

Github et Blog de référence à HackSys Extrem Vulnerable Driver (HEVD)

- Github HEVD: https://github.com/hacksysteam/HackSysExtremeVulnerableDriver?tab=readme-ov-file
- Exploit HEVD win10 Arbitrary Read: https://ommadawn46.medium.com/windows-kernel-exploitation-hevd-on-windows-10-22h2-b407c6f5b8f7
- Github Exploit HEVD win10 Arbitrary Read: https://github.com/ommadawn46/HEVD-Exploit-Win10-22H2-KVAS?tab=readme-ov-file
- Explication table HalDispatch pour mieux comprendre Exploit HEVD win10 Arbitrary Read: https://medium.com/@WaterBucket/understanding-haldispatchtable-for-windows-vulnerability-research-exploit-development-417568ec60c2